EduTrack SA

Digital Academic Monitoring & Engagement System

Software Requirements Specification (SRS)

Document	Details
Module	SFG117V / ISE117V — Advanced Diploma in Computer Science
Group	Royal Hackers
Version	2.0 — Full Requirements Specification
Date	April 2026
Examiner	Dr. HD Masethe

1. Stakeholder Analysis

Four stakeholder groups interact with EduTrack SA. Each has distinct goals, pain points, and interaction patterns that directly shape the system requirements.

Stakeholder	Role	Primary Goal	Pain Point Addressed	Success Metric
Learner (Grade 8–12, age 14–18)	End user — accesses content, takes quizzes, tracks own progress	Improve academic performance; understand own weaknesses; stay engaged outside class hours	No visibility into own progress; intermittent internet; no formative assessment feedback	Quiz completion rate ≥70%; learner satisfaction ≥4/5 in UAT
Teacher	Content creator, quiz reviewer, class performance monitor	Reduce assessment workload; identify struggling learners early; deliver relevant content efficiently	10–15 hours/week on manual assessment creation; no class-wide performance visibility	Workload reduced by 30–40%; at-risk learners flagged automatically
School Management (Principal / Deputy)	Oversight — views aggregate school performance and resource usage	Data-driven decisions on resource allocation; track improvement against DBE targets	No aggregate view of digital learning activity; cannot identify systemic gaps	Management dashboard used weekly; reports generated correctly
Tech Admin	System administration — manages users, roles, system settings, monitors AI logs	Keep system running; manage onboarding; ensure compliance and security	No central tool to manage school users or audit AI usage	User management tasks completable without developer help; AI logs accessible

2. Functional Requirements (FR-001 – FR-020)

Each requirement is specified with a unique ID, title, description, actor, preconditions, postconditions, and priority. Developers must implement ALL must-have requirements before sprint 4.

ID	Title	Description	Actor	Precondition	Postcondition
FR-001	User Registration	A new user (learner, teacher, or management) can create an account by providing name, email, password, role, and school. System validates email format and password strength (≥8 chars, 1 number). Password hashed with bcrypt (12 rounds).	All	No account exists for the email	Account created; welcome email sent; JWT tokens issued
FR-002	User Login	Registered user enters email and password. System verifies credentials. On success: issue JWT access token (1 hr) and refresh token (7 days). After 3 failed attempts: lock account for 30 minutes.	All	Valid account exists	JWT tokens stored; user redirected to role-specific dashboard
FR-003	Browse Learning Content	Learner can view list of available subjects and access content uploaded by their teacher. Content displayed as cards with title, subject, upload date.	Learner	User is authenticated; teacher has uploaded content	Content list renders within 1.5s; learner can open any content item
FR-004	Download Content for Offline	Learner can download content to device storage via Service Worker cache. A 'Download' button triggers PWA caching. Cached items show an offline-available indicator.	Learner	User is authenticated; browser supports Service Worker	Content cached in IndexedDB; accessible without internet connection
FR-005	View Learner Dashboard	Learner sees: total points, current streak, longest streak, subject-wise average scores, recent quiz history (last 5), leaderboard rank.	Learner	User is authenticated as learner	Dashboard renders with real data in <1.5s
FR-006	Take Quiz	Learner opens a published quiz. Questions displayed one at a time (or all on one page for short quizzes). MCQ answers selectable by tap/click. Short-answer typed. Auto-save to IndexedDB every 30 seconds.	Learner	Quiz is in PUBLISHED state; learner has not already submitted	Quiz in progress; answers auto-saved; timer running if applicable
FR-007	Submit Quiz & Auto-Score	On submission, MCQ answers are compared to correct_answer field in DB. Score calculated as (correct/total)*100. Score stored in SUBMISSION	Learner	Quiz in progress; all questions answered or skipped	Score stored; SUBMISSION record created; learner dashboard updated

		table. Result page shows score, correct answers, and explanations.			
FR-008	Track Personal Progress	Learner views per-subject score history as a line chart. Can see improvement over time. Subject breakdown shows average score, quizzes attempted, best score.	Learner	At least 1 quiz submitted	Charts render from SUBMISSION data; subject breakdown correct
FR-009	Earn Points & Streaks	On quiz submission: award points (10 per quiz + bonus for score ≥80%). Increment daily streak if user was active yesterday. Reset streak if missed a day. Store in GAMIFICATION_PROFILE.	Learner	Quiz submitted successfully	GAMIFICATION_PROFILE updated; streak shown on dashboard
FR-010	View Leaderboard	Learner can view class leaderboard ranked by total_points from GAMIFICATION_PROFILE. Top 10 shown. Own rank always shown even if outside top 10.	Learner	User authenticated; class has >1 learner with points	Leaderboard renders correctly; own rank highlighted
FR-011	Upload Learning Content	Teacher uploads file (.txt, .pdf, .docx, .jpg, .png; max 10MB) for a specific subject. System validates file type and size. File stored in Supabase Storage. Content record created in CONTENT table with teacher_id, subject_id, title.	Teacher	User authenticated as teacher	File stored; CONTENT record created; available for AI quiz generation
FR-012	Trigger AI Quiz Generation	Teacher clicks 'Generate Quiz' on a content item. Backend extracts text, builds prompt, calls LLM API (GPT-4o-mini). Returns 5–10 questions as structured JSON. Logged in AI_LOG table.	Teacher	CONTENT record exists with non-empty text	AI-generated QUIZ record in DRAFT state; questions in QUESTION table
FR-013	Review & Edit AI Quiz	Teacher can view all AI-generated questions. Can edit prompt text, edit options, change correct answer, delete individual questions, or add manual questions. Cannot publish without reviewing each question.	Teacher	Quiz in DRAFT or UNDER REVIEW state	All edited questions saved; quiz ready for approval
FR-014	Publish Quiz to Class	Teacher marks quiz as APPROVED, then PUBLISHED. Learners in the school can then see and take the quiz. Teacher can set an optional	Teacher	Quiz in APPROVED state; at least 3 questions	Quiz status = PUBLISHED; visible to learners of that school/subject

		deadline.			
FR-015	Monitor Class Performance	Teacher dashboard shows: class average score per quiz, individual learner scores per quiz, subject performance trends, quiz completion rates.	Teacher	At least 1 quiz has submissions	Dashboard loads with correct aggregated data in <2s
FR-016	At-Risk Learner Flagging	System automatically flags any learner whose last 2 consecutive quiz scores in any subject are below 50%. Flagged learners shown with red indicator on teacher dashboard.	System (automated)	Learner has ≥2 quiz submissions in a subject	At-risk flag stored in DB; visible on teacher dashboard
FR-017	Log Intervention Notes	Teacher can add a text note against a flagged at-risk learner. Notes saved to INTERVENTIONS table with timestamp and teacher_id.	Teacher	Learner flagged as at-risk	Intervention note saved; visible in learner detail view
FR-018	School Management Dashboard	Management user sees: school-wide average score, subject performance breakdown, number of active learners, number of at-risk learners, quiz activity over time (weekly chart).	Management	User authenticated as management	Dashboard renders aggregate data correctly
FR-019	Offline Quiz Submission Sync	When learner submits a quiz while offline, submission stored in SYNC_QUEUE (IndexedDB). When connection restored, Service Worker background-syncs all queued submissions to backend automatically.	Learner	Quiz completed offline; SYNC_QUEUE has pending entries	All submissions sent to backend; SYNC_QUEUE cleared; scores calculated
FR-020	User Management (Admin)	Tech admin can: view all users in the school, activate/deactivate accounts, reset passwords, assign/change roles, export user list. School EMIS number required at setup.	Tech Admin	User authenticated as admin	User record updated in Supabase; changes reflected immediately

3. Non-Functional Requirements (NFR-001 – NFR-011)

ID	Category	Requirement	Measurable Target	Test Method
NFR-001	Performance	Page load time on 3G connection for all core pages	< 1.5 seconds (Lighthouse Performance score ≥85)	Lighthouse CI on every PR
NFR-002	Performance	REST API response time for all non-AI endpoints	< 500ms (p95) under normal load	Postman + k6 load test
NFR-003	Performance	AI quiz generation end-to-end response time (content upload to questions returned)	< 30 seconds per generation request	TC-AI-01 automated test
NFR-004	Availability	System uptime per calendar month (Vercel + Supabase combined)	≥ 99.5% (max 3.6 hrs downtime/month)	Prometheus uptime monitoring; monthly SLA report
NFR-005	Security	Authentication token management — JWT access and refresh token expiry	Access token: 1 hour; Refresh token: 7 days; Rotation on every refresh	Auth integration tests; token expiry verification
NFR-006	Security	Data encryption — all stored data and data in transit	AES-256 at rest (Supabase managed); TLS 1.3 in transit (HTTPS enforced; HTTP redirect)	SSL Labs test; Supabase encryption settings audit
NFR-007	Usability	Learner-facing UI comprehensibility — target audience Grade 9 learner (age 14)	Task completion rate > 90% in UAT with learners; SUS score ≥ 70	UAT Session with 20 learners in Sprint 5
NFR-008	Usability	Data-saving mode page weight — for learners in low-bandwidth environments	< 100KB per page in data-saving mode (text-only, lazy-loaded images)	Chrome DevTools Network throttle; Lighthouse byte budget
NFR-009	Reliability	System error rate — percentage of requests resulting in 5xx errors	< 1% of all API requests (measured over 7-day rolling window)	Prometheus error rate alert; ELK Stack log analysis
NFR-010	Scalability	Concurrent user support — simultaneous active learners without performance degradation	≥ 100 concurrent learners; response time stays < 500ms	k6 load test: 100 virtual users, 5-minute ramp
NFR-011	Maintainability	Code quality — test coverage and defect density across backend and frontend codebases	Code coverage ≥ 75% (pytest-cov + Jest); defect density < 4/KLOC (SonarQube)	CI quality gate on every PR; SonarQube dashboard

4. AI-Specific Requirements (AI-01 – AI-08)

These requirements govern the AI quiz generation feature. Every AI requirement must be verified before the system can be considered production-ready. The responsible developer is the backend lead (Nkambule MM).

ID	Requirement	Specification	Measurable Criterion	Fallback
AI-01	Input specification	System accepts text-based learning content. Supported sources: .txt (raw text), .pdf (pdfminer extraction), .docx (python-docx extraction). Minimum 200 words required for meaningful quiz generation. Content stripped of personal data before API call.	Content extraction accuracy > 90% for .txt and .pdf files (verified in TC-AI-01)	If extraction fails: return error to teacher; do not send to LLM
AI-02	Output specification	LLM must return structured JSON array. Each object: { "type": "MCQ|SHORT", "prompt": string, "options": [string] (MCQ only), "correct_answer": string, "explanation": string }. Backend validates JSON schema before presenting to teacher.	Valid JSON returned with no syntax errors; schema validation passes (verified by backend schema checker)	If JSON invalid: retry once; if still invalid, use fallback model; log error
AI-03	Accuracy & relevance	Generated questions must be directly derived from the uploaded content. Questions must not introduce concepts not present in the content. Teacher rates each question on a 5-point relevance scale during review.	Teacher relevance rating ≥ 4/5 for 85% of questions across all UAT sessions (5 teachers, 20 content items)	Questions scoring < 3/5 must be flagged for deletion or edit during review
AI-04	Human oversight mandate	NO AI-generated question may be published to learners without explicit teacher approval. The review interface must require the teacher to act on (approve/edit/delete) every question individually before the Publish button becomes active.	100% of AI-generated questions reviewed before learner access. Publish button disabled until all questions actioned. Verified in TC-AI-01.	N/A — human review is mandatory; cannot be bypassed under any circumstance
AI-05	Fallback & circuit breaker	Primary LLM: OpenAI GPT-4o-mini. On timeout (>30s) or API error: retry once then switch to Groq Llama 3 (fallback). Circuit breaker: after 5 consecutive LLM failures (either model), open circuit for 60 seconds. While circuit open: show manual quiz creation interface immediately.	Fallback triggers automatically; teacher sees manual editor within 2s of circuit open; no 500 error exposed to user. Verified in TC-AI-02.	Manual quiz creation interface always available regardless of AI service status
AI-06	Transparency label	Every quiz generated (even partially) by AI must display the label: 'AI-generated — reviewed by [Teacher Name]' in the quiz header. This label must be stored in the QUIZ table (ai_generated=true, reviewed_by=teacher_id) and rendered on the quiz interface.	Label present on 100% of AI-generated quizzes visible to learners. Verified by UI test TC-05.	If label missing, quiz must not be displayable to learners — backend enforces this constraint

AI-07	Bias & safety checks	Teacher review interface includes a checklist with items: (1) Questions are culturally appropriate, (2) No stereotyping or discriminatory language, (3) Content is age-appropriate for Grade 8–12, (4) No incorrect factual claims. Teacher must tick all 4 before publishing.	Bias checklist completed for 100% of published AI quizzes. Checklist completion stored in DB. Verified in audit log.	If checklist not completed, Publish button remains disabled — same enforcement as AI-04
AI-08	Audit logging	Every call to the LLM API must be logged in the AI_LOG table with: timestamp, teacher_id (hashed), content_id (hashed — no PII), model used (gpt-4o-mini / llama3), prompt token count, response token count, latency_ms, success (boolean), error_code (if applicable). Logs retained for 30 days.	100% of LLM API calls logged. Log retention verified after 30 days. Tech admin can view logs via admin panel. Verified in TC-AI-03.	Logging failure must not block quiz generation — log asynchronously; if log write fails, write to error queue

5. MoSCoW Prioritisation with Rationale

Requirements are prioritised using the MoSCoW method. Developers must complete all MUST HAVE items before sprint 4 begins. SHOULD HAVE items are targeted for sprint 3–4. COULD HAVE items are bonus features if time permits.

5.1 Must Have (Sprint 1–3)

Req ID	Feature	Rationale	Sprint Target
FR-001, FR-002	Registration & Login	No feature works without authentication. JWT, bcrypt, RBAC are security foundations. Without this, no user can access anything.	Sprint 1
FR-011	Content Upload	AI quiz generation depends on uploaded content. This is the entry point for all teacher workflows. Without it, no quiz can be generated.	Sprint 2
FR-012, FR-013, FR-014	AI Quiz Generation, Review, Publish	Core unique value proposition of the system. Directly reduces teacher workload. Required for AI integration (LO L.1) and AI responsible use rubric criterion.	Sprint 4
FR-006, FR-007	Take Quiz & Auto-Score	Primary learner action. Without quiz-taking, the system has no educational value. Auto-save (IndexedDB) is mandatory for offline-first architecture.	Sprint 3
NFR-005, NFR-006	JWT Auth + AES-256 + TLS 1.3	System handles learner data (minors). POPIA compliance is non-negotiable. Encryption and JWT are baseline security requirements.	Sprint 1
AI-04, AI-05	Human oversight + Circuit breaker	AI-generated content must never reach learners without teacher review. Circuit breaker prevents system failure when LLM API is unavailable. Both are safety-critical.	Sprint 4

5.2 Should Have (Sprint 3–4)

Req ID	Feature	Rationale	Sprint Target
FR-005, FR-008	Learner Dashboard & Progress Tracking	Key differentiator: learner agency over own progress. Directly addresses the pain point of learners having no visibility into their performance. Required for LO E.1 (learner-centric requirements).	Sprint 3
FR-009, FR-010	Gamification & Leaderboard	Drives daily engagement without additional infrastructure. Research shows gamification increases daily active usage. Not required for core academic function but improves adoption.	Sprint 3–4
FR-015, FR-016,	Teacher Dashboard, At-Risk Flagging,	Enables the early intervention use case that directly impacts learner	Sprint 3

FR-017	Intervention Notes	outcomes. Automated at-risk detection reduces the need for teachers to manually scan all scores.	
FR-019	Offline Sync Queue	Critical for Quintile 1–3 school context with intermittent connectivity. Without this, quiz submissions are lost when connection drops. Technically complex — requires Service Worker background sync.	Sprint 3
NFR-001, NFR-008	Page load <1.5s + Data-saving mode	Target users have low-end devices and limited data. Performance and data efficiency are more important here than in typical web apps — directly impacts accessibility for the target community.	Sprint 3–4

5.3 Could Have (Sprint 5 — if time allows)

Req ID	Feature	Rationale	Sprint Target
FR-018	School Management Dashboard	Useful for the management stakeholder but not critical for the core educational function. Can be delivered as read-only analytics views using existing data.	Sprint 5
FR-020	Tech Admin User Management	Useful for school IT admin but Supabase admin panel can serve as a stopgap for the semester prototype. A custom admin UI is a phase 2 feature.	Sprint 5
AI-07	Bias & Safety Checklist UI	The checklist logic is important but the UI can initially be a simple checkbox modal. A more sophisticated content-safety filter (e.g. OpenAI Moderation API) is a phase 2 enhancement.	Sprint 4–5

5.4 Won't Have (Phase 1)

The following are explicitly out of scope for the semester prototype and must not be developed in Sprint 1–5:

- Native iOS/Android app (React Native or Flutter) — web PWA covers the use case
- Integration with DBE national systems or any external SIS
- Custom AI model training or fine-tuning — pre-trained LLM APIs only
- Parent/guardian portal — planned for Phase 2
- Real-time video conferencing or live class features
- Predictive analytics or learning pathway recommendations
- Offline AI quiz generation
- SMS notifications

6. Requirements Traceability Matrix

This matrix links each requirement to its design element (component/module/table), its test case, and the learning outcome it satisfies. Developers use this to ensure no requirement is implemented without a corresponding test, and no test exists without a requirement.

Req ID	Requirement Title	Design Element	DB / API Element	Test Case	LO
FR-001	User Registration	AuthService.register(); /api/auth/register POST	USER table; bcrypt hash; Supabase Auth	TC-01	H.1, E.1
FR-002	User Login	AuthService.login(); JWT middleware; /api/auth/login POST	USER table; JWT access+refresh tokens	TC-02	H.1
FR-003	Browse Content	ContentService.list(); ContentCard component; /api/content GET	CONTENT table; SUBJECT table	TC-03	F.1, E.1
FR-004	Offline Download	ServiceWorker.js (cache strategy); OfflineIndicator component	IndexedDB: content-cache store	TC-04	J.1, L.1
FR-005	Learner Dashboard	DashboardPage; ProgressChart; StreakDisplay; /api/learner/stats GET	GAMIFICATION_PROFILE; SUBMISSION table aggregation	TC-05	E.1
FR-006	Take Quiz	QuizPage; QuestionDisplay; AutoSaveHook; /api/quiz/{id} GET	QUIZ, QUESTION tables; IndexedDB quiz-progress store	TC-06	G.1
FR-007	Submit & Score	QuizService.submit(); ScoreEngine; /api/submissions POST	SUBMISSION, ANSWER tables; scoring algorithm	TC-07	G.1, C.1
FR-008	Progress Tracking	ProgressPage; SubjectBreakdown; LineChart component	SUBMISSION table (GROUP BY subject_id)	TC-05	E.1
FR-009	Points & Streaks	GamificationService; PointsCalculator; /api/gamification PATCH	GAMIFICATION_PROFILE table; trigger on SUBMISSION INSERT	TC-08	L.1
FR-010	Leaderboard	LeaderboardPage; /api/leaderboard GET	GAMIFICATION_PROFILE ORDER BY total_points DESC	TC-09	E.1
FR-011	Content Upload	UploadForm; ContentService.upload(); /api/content POST	CONTENT table; Supabase Storage bucket	TC-10	E.1, I.1
FR-012	AI Quiz Generation	AIQuizService.generate(); LLMClient (GPT-4o-mini); CircuitBreaker	QUIZ, QUESTION tables; AI_LOG table	TC-AI-01	L.1, K.1
FR-013	Review AI Quiz	QuizReviewPage; QuestionEditor; /api/quiz/{id}/questions PATCH	QUESTION table UPDATE; QUIZ status = UNDER_REVIEW	TC-AI-01	L.1
FR-	Publish Quiz	PublishButton;	QUIZ status =	TC-11	G.1

014		/api/quiz/{id}/publish POST	PUBLISHED; published_at timestamp		
FR-015	Teacher Dashboard	TeacherDashboard; ClassStats; /api/teacher/dashboard GET	SUBMISSION aggregation; QUIZ join SUBMISSION	TC-12	C.1
FR-016	At-Risk Flagging	AtRiskService.evaluate(); Cron job (daily); AtRiskBadge component	SUBMISSION (last 2 per subject); at_risk flag in USER	TC-13	C.1
FR-017	Intervention Notes	InterventionForm; /api/interventions POST	INTERVENTIONS table (user_id, teacher_id, note, timestamp)	TC-14	E.1
FR-018	Management Dashboard	ManagementDashboard; /api/school/analytics GET	Aggregate view over SUBMISSION, USER, QUIZ by school_id	TC-15	E.1
FR-019	Offline Sync	SyncQueueService; BackgroundSync API; /api/sync POST	SYNC_QUEUE table; IndexedDB sync-queue store	TC-16	J.1, H.1
FR-020	User Management	AdminPanel; UserService.manage(); /api/admin/users CRUD	USER table; Supabase Auth admin API	TC-17	H.1
AI-08	AI Audit Logging	AILogService; async log writer; /api/admin/ai-logs GET	AI_LOG table (see AI-08 spec)	TC-AI-03	L.1, H.1
NFR-001	Page Load <1.5s	Next.js code splitting; image optimisation; lazy loading	N/A (frontend build config)	Lighthouse CI	C.1
NFR-004	≥99.5% Uptime	Vercel serverless deployment; Supabase HA PostgreSQL	Cloud infrastructure (external SLA)	Prometheus monitor	H.1

7. Use Case Diagrams

The following diagrams show all system use cases grouped by actor. Each use case maps to one or more functional requirements listed in Section 2.

7.1 Learner Use Cases

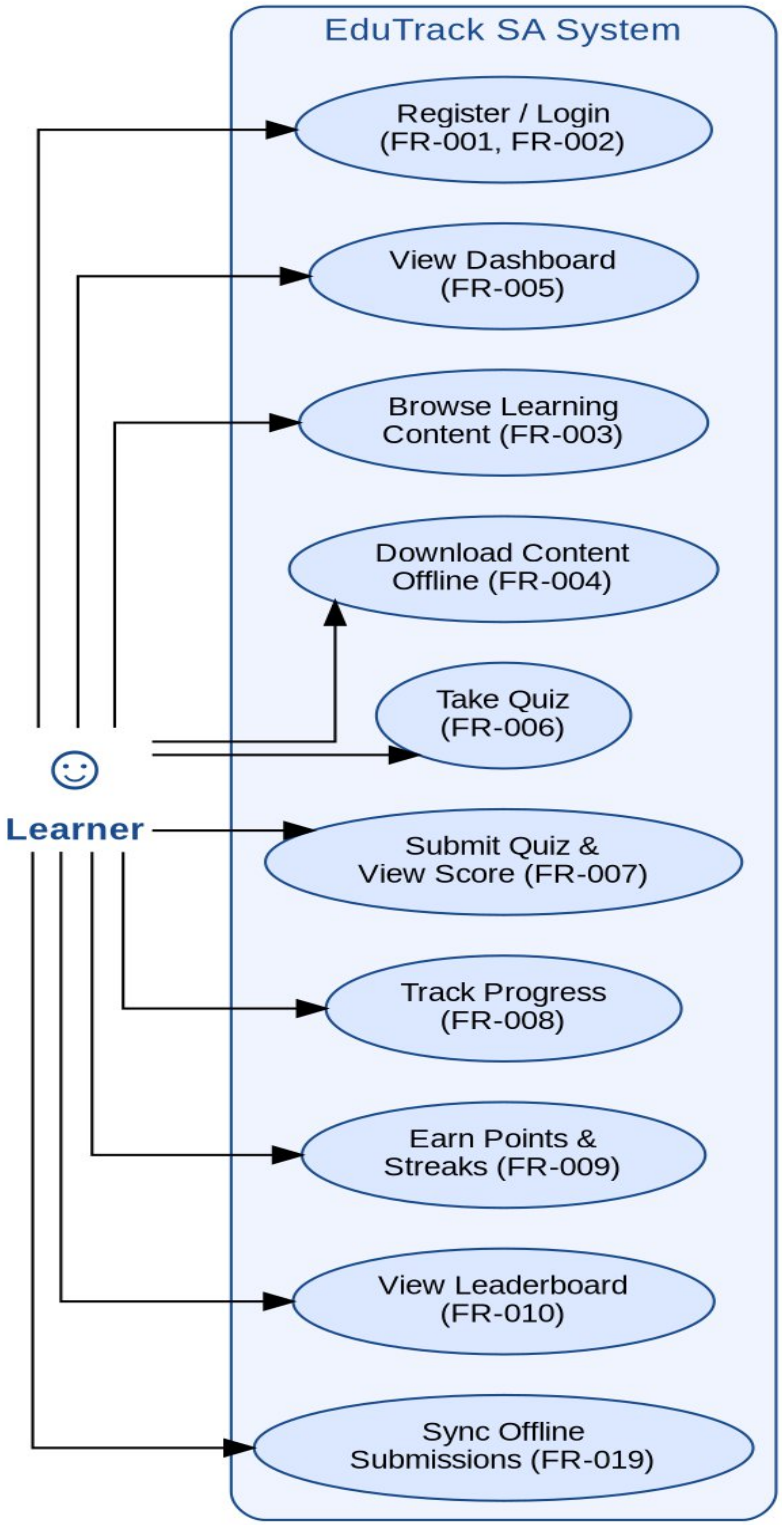

Figure 1: Learner Use Case Diagram — FR-001 to FR-010, FR-019

7.2 Teacher Use Cases

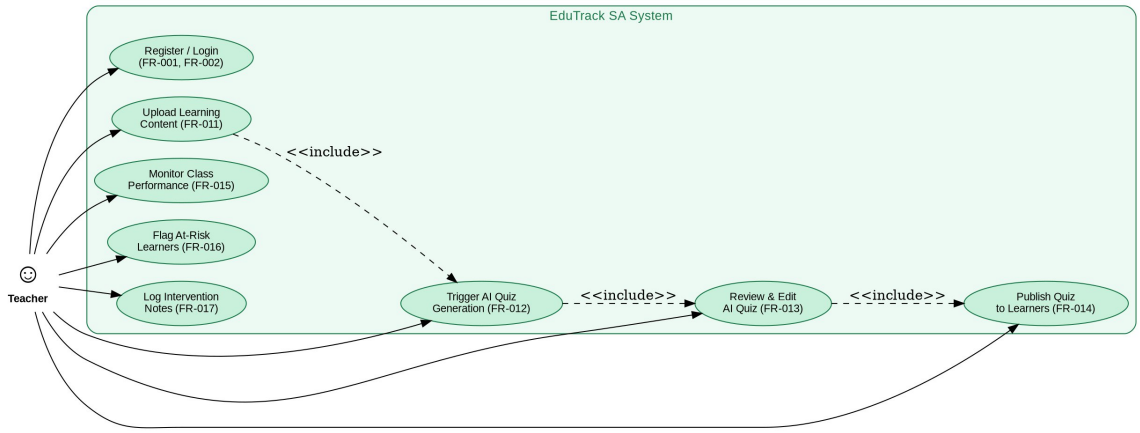

Figure 2: Teacher Use Case Diagram — FR-001, FR-011 to FR-017

7.3 School Management & Tech Admin Use Cases

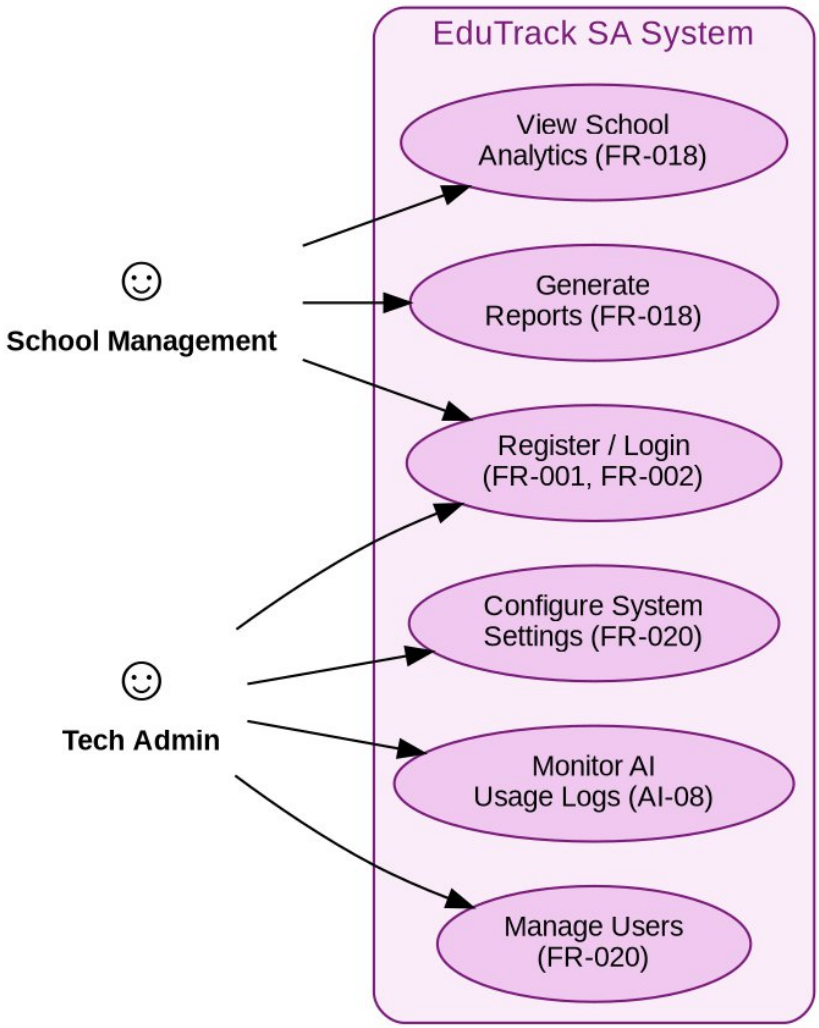

Figure 3: Management & Admin Use Case Diagram — FR-018, FR-020, AI-08

8. Activity Diagrams

Activity diagrams show the step-by-step logic of key workflows including decisions, parallel paths, and error handling. Frontend developers should use these to design screen flows. Backend developers should use these to design API endpoint logic.

8.1 User Registration & Login Flow (FR-001, FR-002)

Backend note: Implement account lockout counter in Redis or Supabase. JWT issuance handled by Supabase Auth. Frontend note: Inline validation on every field; do not wait for API call to show format errors.

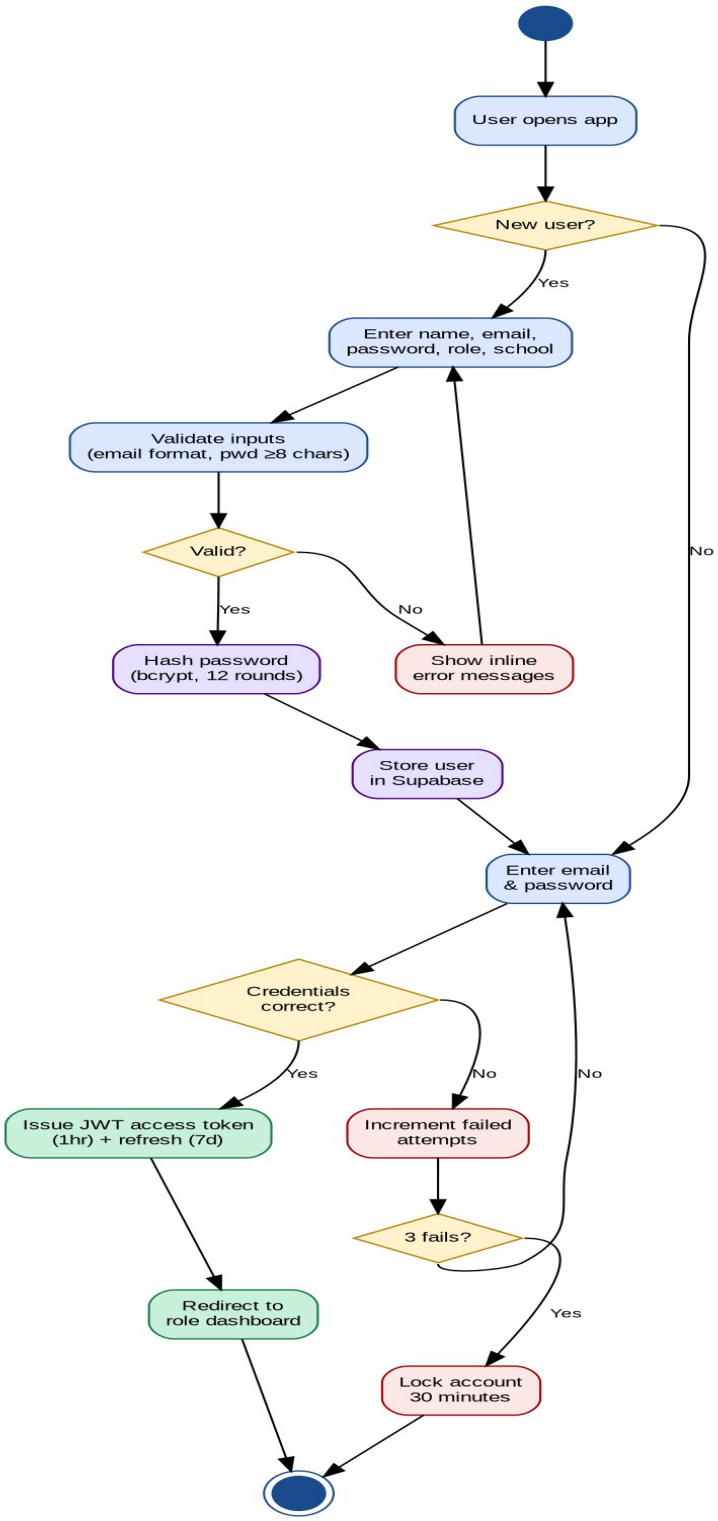

Figure 4: Authentication Activity Diagram — FR-001, FR-002, NFR-005, NFR-006

8.2 AI Quiz Generation Flow (FR-012, FR-013, FR-014)

Backend note: Implement CircuitBreaker as a Python class wrapping LLMClient. Use Redis for failure counter. Frontend note: Show progress spinner during generation; display error toast on circuit open; redirect to manual editor.

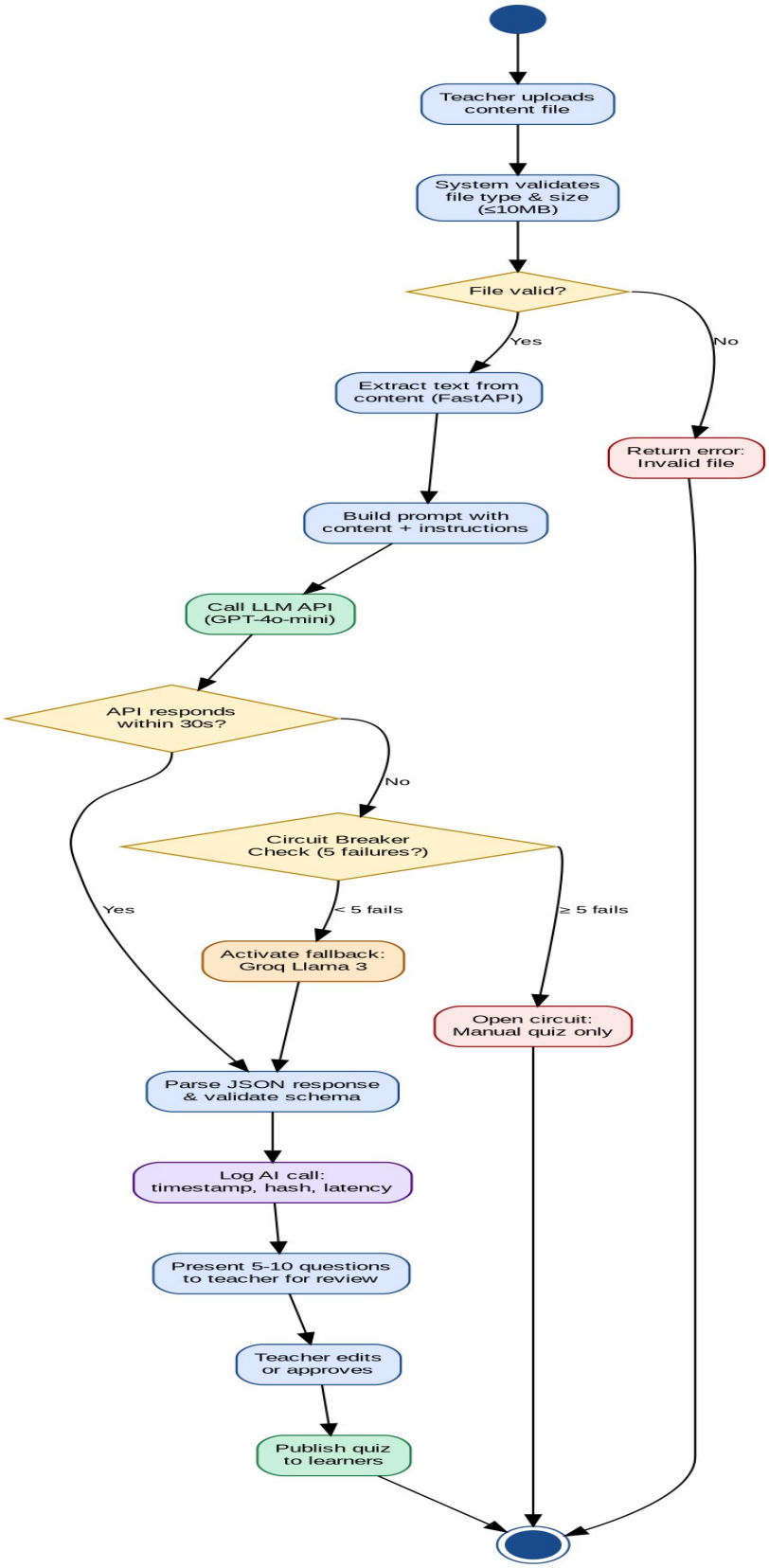

Figure 5: AI Quiz Generation Activity Diagram — FR-012 to FR-014, AI-01 to AI-08

8.3 Learner Takes Quiz & Submits (FR-006, FR-007, FR-019)

Backend note: POST /api/submissions must be idempotent (use submission_id UUID generated client-side). Frontend note: Auto-save hook runs every 30s using setInterval; saves to IndexedDB only — never to server mid-quiz.

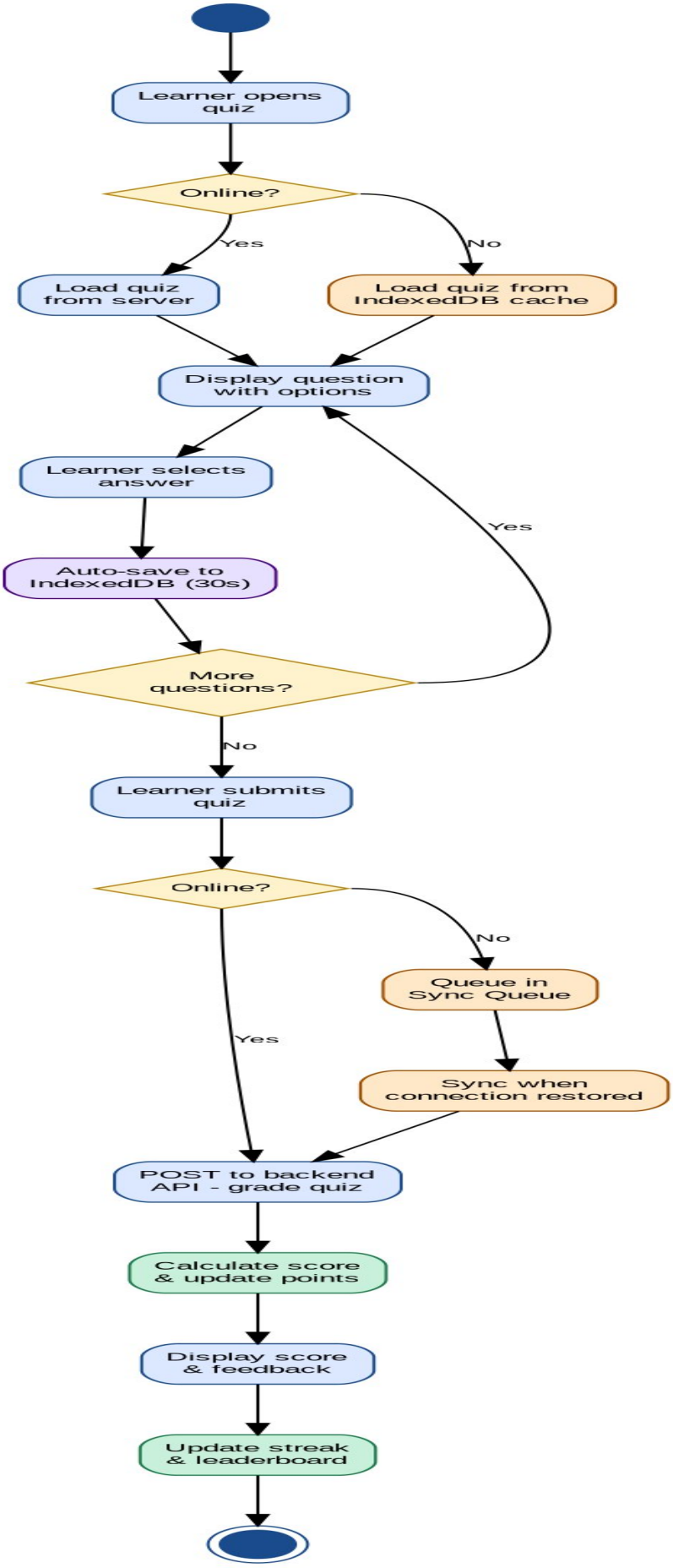

Figure 6: Learner Quiz Activity Diagram — FR-006, FR-007, FR-019, NFR-001

8.4 Teacher Dashboard & At-Risk Flagging (FR-015, FR-016, FR-017)

Backend note: At-risk evaluation runs as a daily cron job AND on every quiz submission (real-time flag). Frontend note: Red badge on learner card; clicking opens learner detail drawer with quiz history chart.

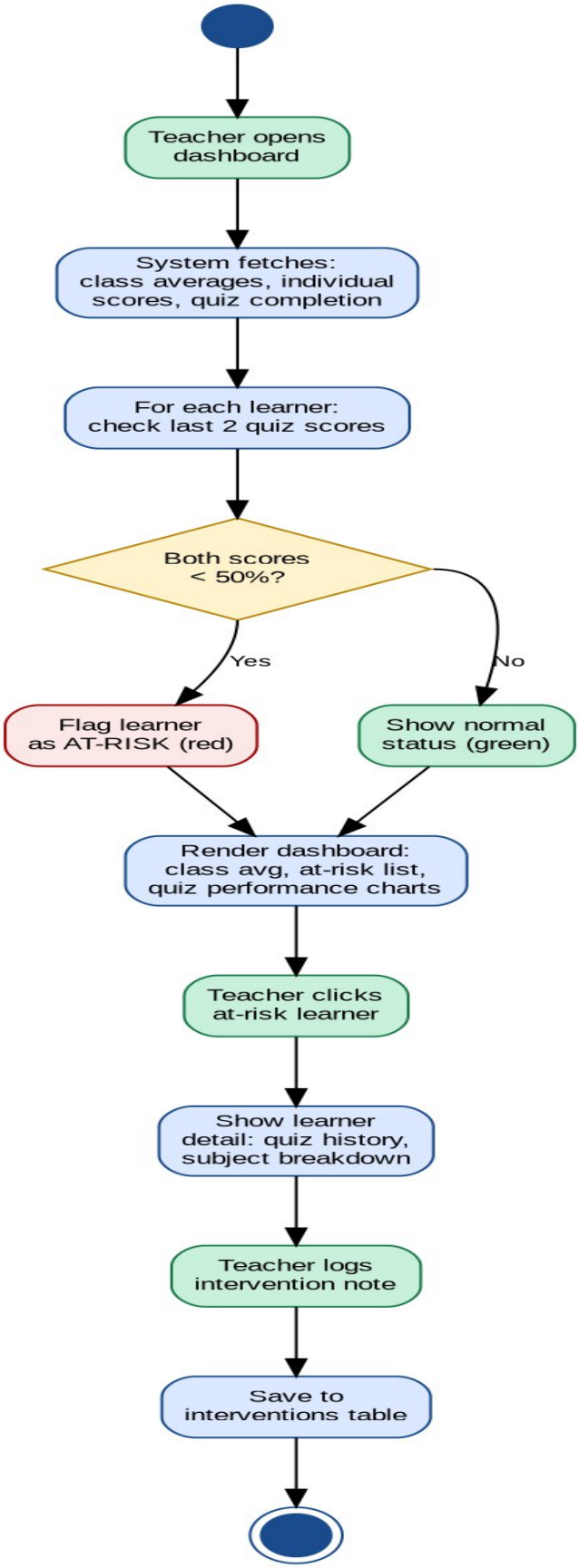

Figure 7: Teacher Dashboard Activity Diagram — FR-015 to FR-017, NFR-002

9. State Machine Diagrams

State machines define all valid states an entity can be in and the transitions between them. Developers must enforce these states in both the database (status enum columns) and the API (reject invalid state transitions with 422 error).

9.1 Quiz Lifecycle State Machine (FR-012 to FR-014)

Developer note: The QUIZ.status column must be an enum: DRAFT | UNDER_REVIEW | APPROVED | PUBLISHED | ARCHIVED. API must validate that transitions are sequential (e.g. cannot go from DRAFT directly to PUBLISHED). Add a DB constraint or service-layer guard.

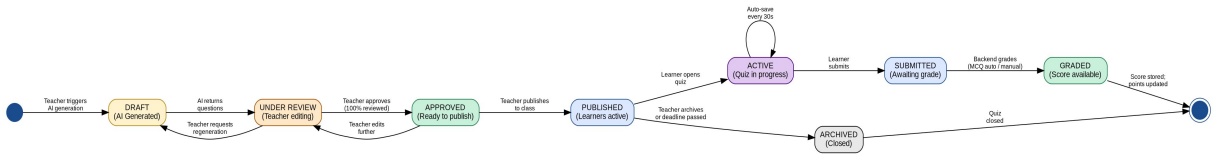

Figure 8: Quiz Lifecycle State Machine — FR-012, FR-013, FR-014

9.2 User Session / Authentication State Machine (FR-001, FR-002)

Developer note: Implement silent token refresh in a Next.js API route called by an Axios interceptor on every 401 response. Store failed_attempts in Redis with TTL=30 minutes. NEVER store failed attempts in the main DB — it creates a write bottleneck.

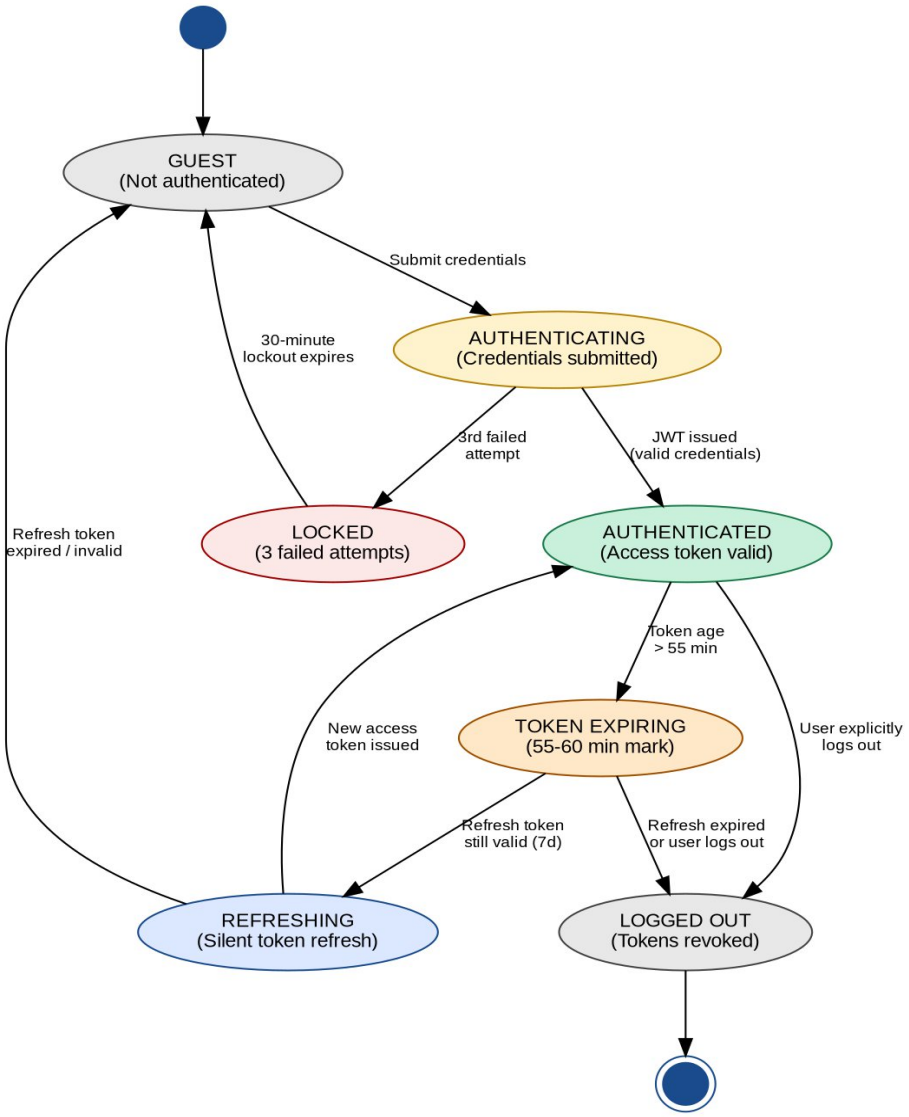

Figure 9: User Session State Machine — FR-002, NFR-005

9.3 Offline Sync State Machine (FR-019)

Developer note: Use the Background Sync API (Service Worker) — register a sync tag 'quiz-sync' on queue. IndexedDB store 'sync-queue' holds {submission_id, quiz_id, answers[], timestamp}. On sync event, iterate queue and POST each item. Remove from queue only on HTTP 200/201 response.

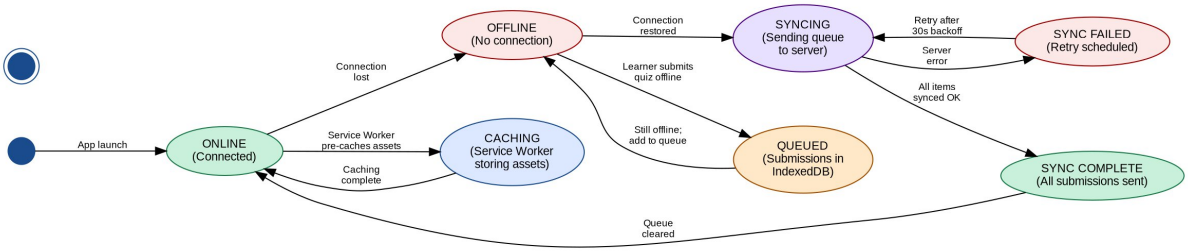

Figure 10: Offline Sync State Machine — FR-019, J.1

10. Business Process Diagram — Content Upload to Quiz Publication

This swimlane diagram shows the end-to-end business process across Teacher, Backend API, and AI Service. Use this to understand the inter-system handoffs and where to add error handling.

Key handoffs for developers: (1) Teacher → Backend: multipart/form-data POST. (2) Backend → AI Service: JSON POST with extracted text + system prompt. (3) AI Service → Backend: structured JSON array. (4) Backend → Teacher: questions array via WebSocket or polling. (5) Teacher → Backend: PATCH /api/quiz/{id}/publish.

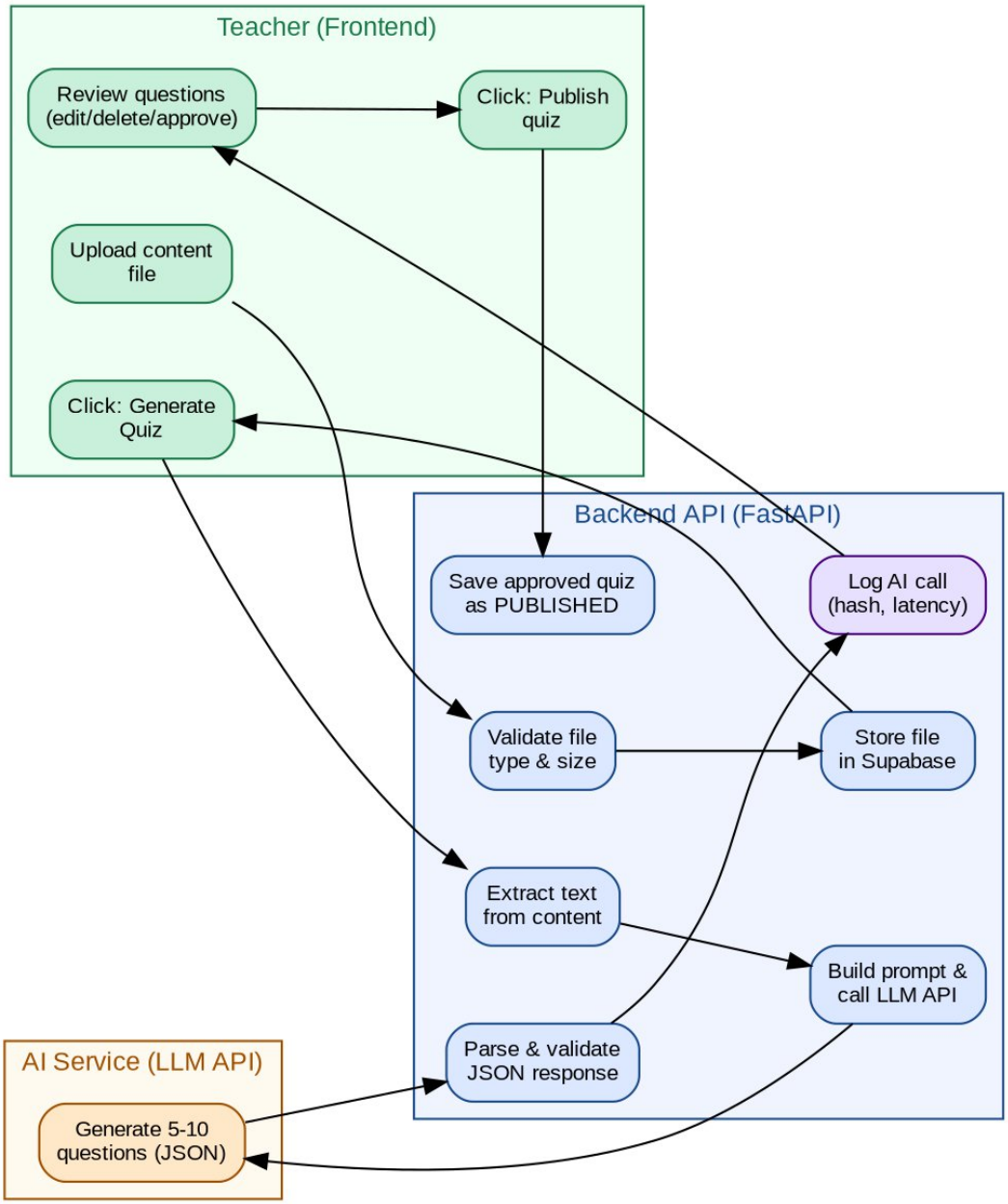

Figure 11: Business Process Swimlane — FR-011 to FR-014, AI-01 to AI-08

11. Developer Implementation Notes

11.1 Backend (FastAPI) — Key Implementation Decisions

Area	Implementation Guidance
Project structure	Use feature-based modules: /app/auth/, /app/content/, /app/quiz/, /app/ai/, /app/gamification/, /app/admin/. Each module has: router.py, service.py, schemas.py, models.py.
Database (Supabase)	Use Supabase Python client for all DB operations. Enable Row-Level Security (RLS) on USER, SUBMISSION, CONTENT, QUIZ tables. Policy: users can only SELECT/INSERT their own school's data. Enable PITR (point-in-time recovery) from Day 1.
JWT middleware	Create FastAPI dependency: get_current_user(token: str = Depends(oauth2_scheme)). Verify JWT signature using Supabase JWT secret. Inject user object into all protected routes. Reject expired tokens with 401.
AI service (LLMClient)	Build LLMClient class with: generate(content: str) → list[Question]. Primary: OpenAI API. Fallback: Groq API. CircuitBreaker wraps both. System prompt: constrain output to JSON only. Validate response with Pydantic schema before returning to caller.
Offline sync endpoint	POST /api/sync accepts a batch of submissions. Use database transactions: all-or-nothing per batch. Return 207 Multi-Status if some succeed and some fail. Client retries only failed items.
CI/CD pipeline	GitHub Actions workflow: (1) lint with ruff, (2) run pytest with coverage ≥75%, (3) SonarQube scan, (4) Docker build, (5) deploy to Render on merge to main. PR cannot merge if any step fails.

11.2 Frontend (Next.js PWA) — Key Implementation Decisions

Area	Implementation Guidance
Project structure	Use Next.js App Router. Pages: /app/(auth)/login, /app/(auth)/register, /app/learner/dashboard, /app/learner/quiz/[id], /app/teacher/dashboard, /app/teacher/content/upload, /app/teacher/quiz/[id]/review, /app/admin/users.
Auth (JWT handling)	Store access token in memory (NOT localStorage — XSS risk). Store refresh token in httpOnly cookie. Create Axios interceptor to: (1) add Authorization header, (2) on 401: call /api/auth/refresh silently, (3) retry original request with new token.
Service Worker (PWA)	Use next-pwa library. Cache strategy: NetworkFirst for API calls, CacheFirst for static assets, StaleWhileRevalidate for content files. Register BackgroundSync for offline quiz submissions. Test PWA with Lighthouse.
IndexedDB (offline)	Use Dexie.js (IndexedDB wrapper). Stores: content-cache (content items), quiz-progress (in-progress answers), sync-queue (offline submissions). Auto-save hook: useAutoSave(answers, quizId) runs every 30s.
State management	Use Zustand for global state (user session, quiz state, gamification). Use React Query (TanStack Query) for server state (caching API responses, background refetch). Avoid prop-drilling — use context only for theme/auth.
Data-saving mode	Add a user preference toggle in settings. When enabled: (1) disable all images (replace with CSS placeholders), (2) disable animations, (3) load minimal CSS only. Store preference in Supabase USER.data_saving_mode column.

References

Department of Basic Education, South Africa (2023). National Senior Certificate Technical Report 2023. Pretoria: Government Printing Works.

Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Boston: Addison-Wesley.

Pressman, R.S. and Maxim, B.R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). New York: McGraw-Hill.

Sommerville, I. (2016). Software Engineering (10th ed.). Boston: Pearson.

OpenAI (2024). GPT-4o-mini API Documentation. Available at: https://platform.openai.com/docs/models/gpt-4o-mini

Groq (2024). Llama 3 API Documentation. Available at: https://console.groq.com/docs/llama3

Supabase (2024). Supabase Documentation. Available at: https://supabase.io/docs